DATA ENGINEERING PROJECT

Docker Data Pipeline Framework using Apache Airflow, Docker and PostgreSQL

(Implemented on the Olist E-Commerce Dataset — Bronze, Silver, Gold Medallion Architecture)

Project Repository

https://github.com/Jayasrirajendiran/docker-data-pipeline

1. Abstract

This report documents the design, development and implementation of a Data Engineering Project built around the Olist Brazilian E-Commerce public dataset from Kaggle. The objective of the project is to demonstrate a complete, production-style Extract, Transform and Load (ETL) workflow that takes raw, semi-structured CSV files and converts them into clean, governed and business-ready datasets suitable for analytics and reporting.

The pipeline is orchestrated using Apache Airflow and containerised with Docker so that the webserver, scheduler and PostgreSQL services run in a reproducible environment. Data moves through Bronze, Validation and Silver stages. SCD Type 1 and SCD Type 2 then run in parallel on independent customer and product dimensions; business transformation waits for both, builds the joined analytical dataset, and produces Gold summaries. Incremental order logic is implemented inside ingestion rather than as a separate task. PostgreSQL loading is followed by one Metadata task and one final Audit task.

The project illustrates how open-source tools alone- Python, Pandas, Apache Airflow, Docker and PostgreSQL, can be combined to build a scalable, maintainable and auditable data platform without depending on proprietary or cloud-only services, making the solution portable and cost-effective for small teams and learning environments.

2. Introduction

Modern organisations generate data continuously from multiple operational systems, including order management, payments, logistics, and customer relationship platforms. This raw data is rarely fit for direct analysis: it is scattered across files, contains duplicates, has inconsistent formats, and lacks the structure required for

reliable reporting. A Data Engineering pipeline exists precisely to close this gap to ingest raw data, cleanse and standardise it, apply business logic, and deliver curated datasets to downstream consumers such as dashboards, machine learning models and business analysts.

This project uses the Olist E-Commerce dataset, a widely used open dataset containing information about orders, customers, products, payments and reviews from a Brazilian marketplace, as a realistic stand-in for a company's operational data. The pipeline automates the full journey of this data from raw CSV ingestion through validation, cleaning, transformation, dimensional modelling and, finally, into a relational data warehouse (PostgreSQL) using Apache Airflow for orchestration and Docker for environment consistency.

The solution follows the Medallion architecture pattern (Bronze, Silver, Gold), a widely adopted industry approach that organises data into progressively refined layers, and adds engineering best practices such as audit logging, metadata tracking, incremental loads and Slowly Changing Dimensions to reflect how real enterprise pipelines are built and operated.

3. Problem Statement

Organisations require automated, reliable and repeatable ETL pipelines to process large volumes of transactional data. Manual processing is slow, error-prone and difficult to audit. Without a structured pipeline, issues such as duplicate records, missing keys and inconsistent data types can propagate into reporting datasets.

This project addresses these problems through a containerised pipeline that ingests, validates, cleans, transforms and loads data through clearly separated tasks. Airflow manages dependencies and retries, while final Metadata and Audit tasks record the produced datasets and overall run outcome.

4. Objectives

- Build a fully automated end-to-end ETL pipeline using Apache Airflow and Docker.
- Implement the Bronze, Silver and Gold layers of the Medallion architecture.
- Perform business transformations such as joins, derived columns, aggregations and ranking.
- Implement Slowly Changing Dimensions SCD Type 1 (overwrite) and SCD Type 2 (historical tracking).
- Implement full and incremental order ingestion in the same ingestion module using a timestamp watermark.
- Load the final Gold layer datasets into PostgreSQL for consumption by BI and analytics tools.
- Generate one consolidated metadata file and one final audit record for pipeline traceability.
- Containerise the entire stack with Docker for portability and environment consistency.

5. Technology Stack

The project is built entirely on open-source technologies, summarised below.

Component	Technology	Purpose
Programming Language	Python	Core language for all ETL scripts and transformation logic
Orchestration	Apache Airflow	Scheduling, dependency management and monitoring of pipeline tasks

Component	Technology	Purpose
Containerisation	Docker & Docker Compose	Isolated, reproducible runtime environment for all services
Data Processing	Pandas	Data cleaning, validation and transformation
Database	PostgreSQL	Relational data warehouse for Gold layer datasets
ORM / DB Connectivity	SQLAlchemy	Loading Pandas DataFrames into PostgreSQL
Intermediate Storage	Parquet	Efficient columnar storage for Bronze/Silver/Gold layers
Database Administration	pgAdmin	Verification and querying of loaded data

6. System Architecture

The system follows a layered, container-based architecture. Raw CSV files are read by ingestion, and Airflow runs every later stage as an individual task. Dependencies are sequential through Silver; SCD Type 1 and Type 2 run in parallel because they process independent dimensions. Business transformation begins only after both SCD tasks succeed, followed by Gold, PostgreSQL, Metadata and Audit.

At a high level, the architecture consists of four cooperating components:

1. Orchestration Layer: Apache Airflow, running inside Docker, schedules and monitors all pipeline tasks.
2. Processing Layer: Python and Pandas scripts perform ingestion, validation, cleaning and transformation.
3. Storage Layer: Parquet files hold intermediate Bronze/Silver/Gold data; PostgreSQL holds the final curated datasets.

4. Governance Layer: One metadata output records final dataset details, and one audit output records the final pipeline status and duration.

The overall data flow through the system is illustrated below:

```
Raw CSV Files
    ↓
Ingestion (Full / Incremental Orders)
    ↓
Bronze → Validation → Silver
    ↓
SCD Type 1 (Customers)  ‖  SCD Type 2 (Products)
    ↓ both complete
Business Transformation → Gold → PostgreSQL → Metadata → Audit
```

7. Medallion Architecture

The Medallion architecture is a data design pattern that organises data into three progressively refined layers, Bronze, Silver and Gold, each with a distinct responsibility. It is widely used in modern data platforms because it provides clear separation between raw, cleaned and business-ready data, making pipelines easier to debug, reprocess and govern.

7.1 Bronze Layer- Raw Data

The Bronze layer stores data exactly as it arrives from the source, with minimal changes, alongside ingestion metadata such as load timestamp and source file name. It acts as the immutable system of record, allowing any downstream layer to be rebuilt from scratch if required.

7.2 Silver Layer - Cleaned and Conformed Data

The Silver layer applies validation, deduplication, type casting and standardisation rules to the Bronze data, producing a clean, conformed dataset that is consistent in structure and free of obvious quality issues.

7.3 Gold Layer - Business-Ready Data

The Gold layer contains fully transformed, aggregated and dimensionally modelled datasets such as customer summaries, product summaries and monthly sales that are ready for direct consumption by BI tools, dashboards and business stakeholders.

8. Project Workflow

The workflow begins when the Olist CSV files are available in the raw directory and ends with Gold and SCD tables loaded into PostgreSQL, followed by final Metadata and Audit tasks.

- Ingestion: source CSV files are copied as full snapshots, while orders use a saved timestamp watermark for incremental processing after the first run.
- Bronze: ingested CSV data is stored as Parquet with load timestamp and source-file metadata.
- Validation: required-key null checks and duplicate-key checks separate valid and rejected records.
- Silver: validated data is deduplicated, standardised and converted to suitable date and numeric types.
- SCD Type 1: the customer dimension keeps the latest record for each customer without preserving overwritten values.
- SCD Type 2: the product dimension preserves changed product versions using effective dates, end dates and an is_current flag.
- Business Transformation: after both SCD tasks finish, orders are joined with current customer and product dimensions, sellers, categories and aggregated payments; derived columns and a window rank are added.

- Gold: business_dataset, customer_summary, product_summary, monthly_summary and seller_summary are produced as analytics-ready Parquet files.
- Load to PostgreSQL: five Gold datasets and two SCD dimensions are loaded as seven PostgreSQL tables.
- Metadata: one final task records dataset, layer, filename, format, row count and update time in pipeline_metadata.csv.
- Audit: the last task records one overall run result, including run ID, timing, duration, status and failed-task information.

9. Module-wise Explanation

Module	Description
Ingestion	Copies source snapshots and performs watermark-based incremental processing for orders.
Bronze	Stores ingested data as Parquet with technical load metadata.
Validation	Checks required keys and duplicates, separating valid and rejected rows.
Silver	Deduplicates, standardises and converts date and numeric columns.
SCD Type 1	Keeps the latest customer record without preserving overwritten values.
SCD Type 2	Maintains product versions with effective dates and a current flag.
Business Transformation	Uses both SCD outputs for joins, lookups, aggregation, derived fields and ranking.
Gold	Builds business and summary datasets for analytics.

Module	Description
Load PostgreSQL	Loads five Gold datasets and two SCD dimensions into PostgreSQL.
Metadata	Writes one final consolidated dataset inventory.
Audit	Writes one final pipeline run outcome after Metadata.

10. Key Concepts

This section explains, in detail, the core engineering concepts that underpin the pipeline from the Bronze layer through to the Audit framework so that the design decisions behind each stage are clear.

10.1 Bronze Layer

The Bronze layer is the entry point of the Medallion architecture. It stores incoming CSV data in its raw form, with no business logic applied, so that the source data is always preserved and can be replayed if a downstream stage needs to be rebuilt.

Each Bronze record is enriched with technical metadata such as ingestion timestamp, batch ID and source file name, which is later used by the audit and metadata frameworks to trace data lineage back to its origin.

10.2 Validation Layer

The Validation layer sits between Bronze and Silver and acts as a quality gate. It checks that incoming data conforms to the expected schema (correct columns and data types), flags null values in mandatory fields, and identifies duplicate records.

Records that fail validation are logged and can be quarantined rather than silently dropped, ensuring that data quality issues are visible rather than hidden inside downstream transformations.

10.3 Silver Layer

The Silver layer takes validated Bronze data and applies cleaning and standardisation rules, trimming whitespace, normalising date formats, casting columns to correct data types, and removing exact duplicates to produce a conformed dataset that is safe to join and aggregate.

10.4 Business Transformation

This stage applies the project's business logic after the SCD dimensions are ready. Orders are joined with the SCD Type 1 customer dimension, the current SCD Type 2 product records, order items, sellers, category lookup data and aggregated payments. Derived fields include order year, order month, delivery days and item total, while a grouped rank supplies the customer order sequence.

10.5 SCD Type 1

SCD Type 1 is applied to the customer dimension. On later executions, existing and current customer snapshots are combined and only the latest row for each customer_id is retained; previous values are overwritten.

10.6 SCD Type 2

SCD Type 2 is applied to products. Tracked attributes such as category, weight and dimensions are compared with the current version. A change closes the old row and inserts a new row with effective_from, effective_to and is_current fields.

10.7 Gold Layer

The Gold layer contains business_dataset plus customer_summary, product_summary, monthly_summary and seller_summary. Customer ranking is stored as a column in customer_summary rather than as a separate table.

10.8 Incremental Loading

Incremental loading is not a separate file or Airflow task. It is implemented inside ingestion.py for orders. The first run performs a full load and saves the latest order_purchase_timestamp; later runs ingest only records newer than that watermark. The downloaded Olist source is static, so a second unchanged run correctly reports zero new orders.

10.9 PostgreSQL Integration

The Gold layer datasets are loaded into PostgreSQL using SQLAlchemy, which provides a Pythonic interface for creating tables and inserting/updating DataFrame contents directly from Pandas. PostgreSQL was chosen for its reliability, strong SQL support and easy integration with BI and reporting tools, and the loaded data is verified using pgAdmin.

10.10 Metadata Framework

A single Metadata task runs after PostgreSQL loading. It scans the project layers and writes pipeline_metadata.csv with the dataset name, layer, filename, format, row count and update timestamp.

10.11 Audit Framework

A single Audit task runs last. It inspects the Airflow run, records SUCCESS or FAILED, lists failed tasks when applicable, and appends one record containing run ID, start time, end time, duration and error message to audit_log.csv.

11. Airflow DAG Explanation

The pipeline is orchestrated by the Airflow DAG olist_data_pipeline with eleven tasks: ingestion, bronze, validation, silver, scd_type1, scd_type2,

business_transformation, gold, load_postgres, metadata and audit. Incremental logic is part of ingestion and therefore does not appear as a separate task.

Dependencies are sequential through Silver. SCD Type 1 and Type 2 then run in parallel because they use different Silver datasets and do not depend on one another. Business transformation waits for both tasks, and the DAG finishes with PostgreSQL, Metadata and Audit. Airflow retries failed tasks individually and displays each state in Graph view.

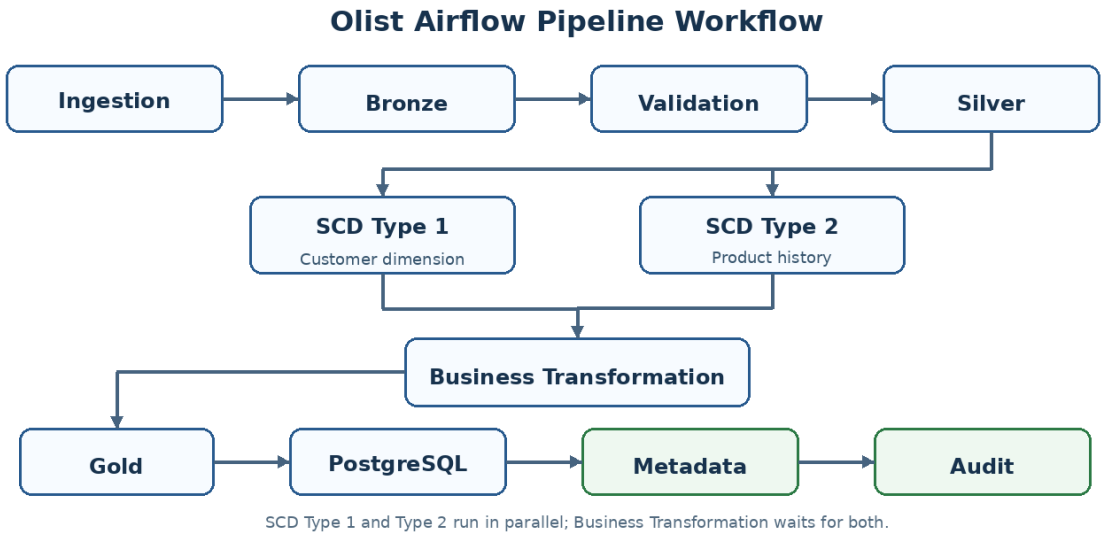


Figure 1: Final Airflow workflow showing parallel SCD tasks and the PostgreSQL → Metadata → Audit sequence.

Airflow's task logs and Audit Log view provide task-level operational details, while the project audit_log.csv stores one consolidated outcome for each complete pipeline run.

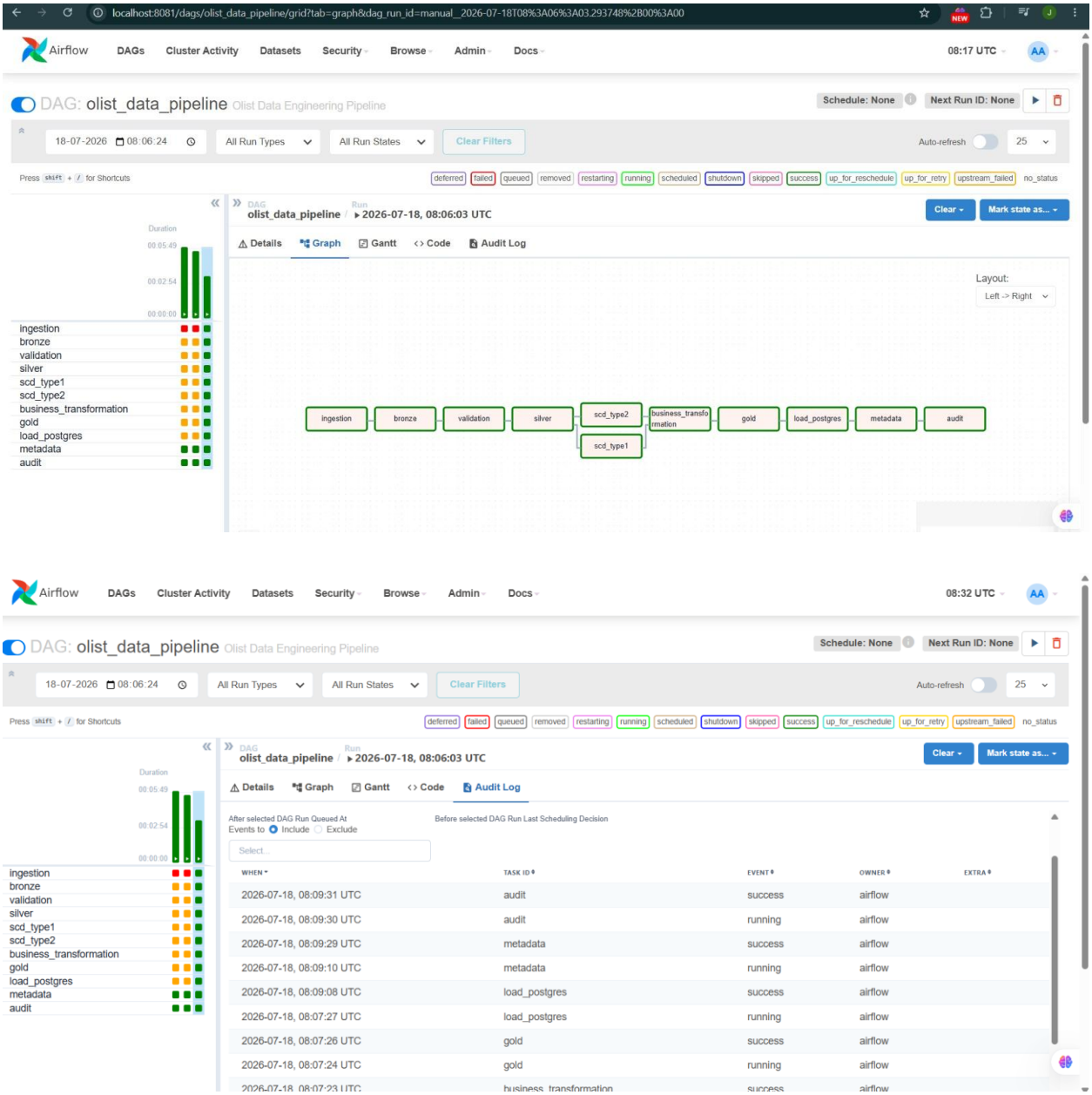


Figure 2: Airflow task-level log and state history used for troubleshooting.

12. Docker Setup

The entire stack, the Airflow webserver, scheduler, metadata database and worker, alongside PostgreSQL, is defined using Docker Compose. This ensures that the pipeline runs identically regardless of the host machine, eliminates dependency

conflicts, and allows the whole environment to be brought up or torn down with a single command.

Running scripts inside the Airflow container (rather than on the host) proved important during development, since file paths referenced by the DAG are relative to the container's filesystem, not the host machine — a lesson captured in the Challenges and Solutions section of this report.

13. Database Design

The Olist PostgreSQL database contains seven reporting tables: business_dataset, customer_summary, product_summary, monthly_summary, seller_summary, customer_dimension and product_dimension_history. The design keeps reporting queries simple while retaining the two SCD dimension outputs.

Table	Key Columns	Description
business_dataset	order_id, customer_id, product_id	Joined analytical dataset (113,425 rows).
customer_summary	customer_unique_id, customer_rank	Customer orders, items, spend and rank (96,219 rows).
product_summary	product_id, product_category_name_english	Product units, orders and sales (32,328 rows).
monthly_summary	order_year, order_month	Monthly orders, customers and revenue (25 rows).
seller_summary	seller_id, seller_city	Seller orders, items and sales (3,095 rows).

Table	Key Columns	Description
customer_dimension	customer_id	Latest customer state from SCD Type 1 (99,441 rows).
product_dimension_history	product_id, is_current	Versioned product history from SCD Type 2 (32,951 rows).

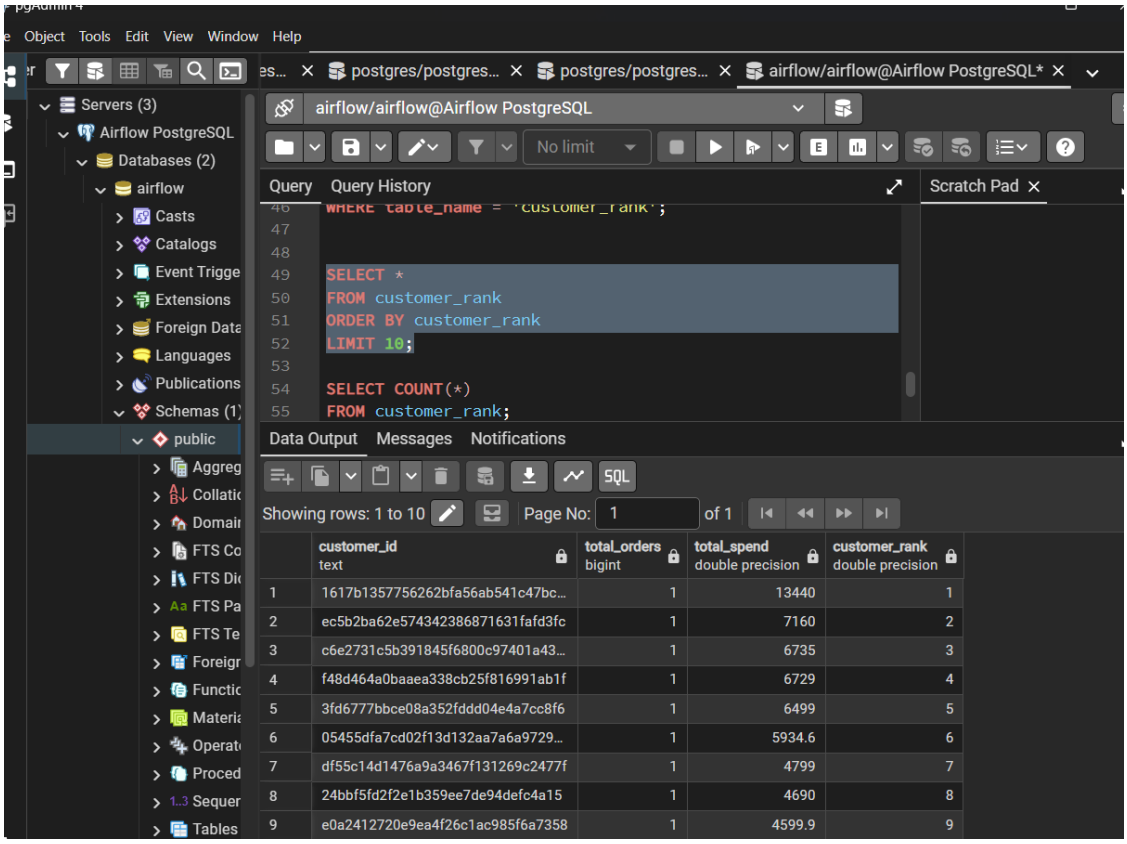
Metadata and audit information is stored in pipeline_metadata.csv and audit_log.csv respectively; these are governance outputs and are not loaded as additional PostgreSQL business tables.

14. SQL Queries with Explanation

The following queries were run in pgAdmin against the loaded PostgreSQL database to verify the correctness and completeness of the Gold layer outputs.

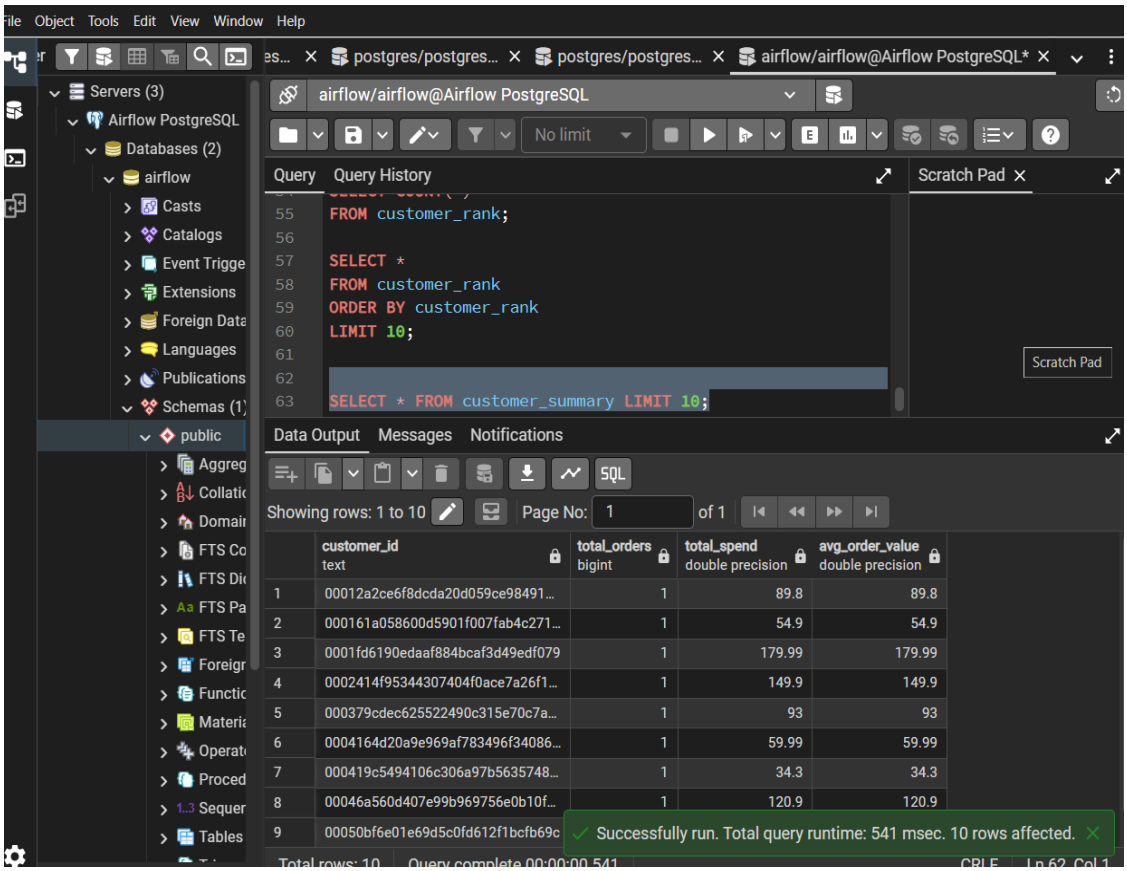
14.1 Top 10 Customers by Rank

The query returns the ten highest-ranked customers from customer_summary by ordering the customer_rank column, confirming that ranking was calculated correctly in the Gold layer.



14.2 Customer Summary Check

The query samples customer_summary to confirm that total_orders, total_items, total_spent and customer_rank were aggregated for each customer_unique_id.

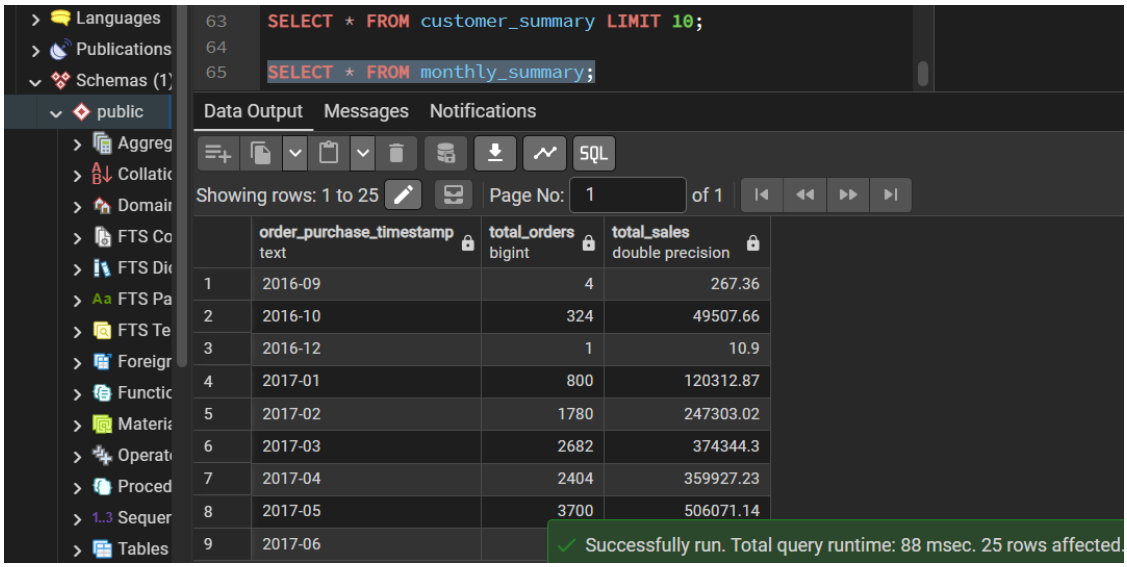


14.3 Monthly Sales Summary

The query retrieves monthly_summary, which contains order counts, distinct customers and item-based revenue grouped by year and month.

14.4 Row Count Verification

A simple row-count check confirms that the master business_dataset table was fully loaded, returning a total of 113,425 rows, matching the expected volume from the source dataset.



15. Testing and Test Cases

Testing was performed at both the module level and the pipeline level to ensure correctness at each stage of the Medallion architecture. The table below summarises the primary test cases exercised during development.

Test Case	Layer / Module	Expected Result
Schema validation on malformed CSV	Validation	Rows with missing mandatory columns are flagged and excluded from Silver.
Duplicate row detection	Validation	Exact duplicate records are identified and removed before Silver.
Null handling in key fields	Silver	Null customer_id / order_id values do not propagate into Gold.
Join correctness	Business Transformation	Row counts after joins match expected cardinality (no fan-out).
SCD Type 1 overwrite	SCD Type 1	Updated dimension value replaces the old value with no duplicate row.
SCD Type 2 history tracking	SCD Type 2	A changed attribute creates a new row with correct effective/expiry dates.

Test Case	Layer / Module	Expected Result
Incremental order check	Ingestion	A repeated unchanged run reports zero new orders and does not append duplicates.
PostgreSQL load verification	Load PostgreSQL	Row counts in PostgreSQL match row counts in the Gold Parquet files.
DAG dependency order	Airflow	SCD Type 1 and Type 2 run in parallel; transformation waits for both.
Final governance outputs	Metadata / Audit	Metadata records final datasets, followed by one overall audit result.

16. Challenges and Solutions

Challenge	Solution
MergeError caused by duplicate metadata columns when joining Silver datasets.	Metadata columns (load timestamp, batch ID) were dropped before performing business joins and re-attached afterwards.
File path errors when scripts were run outside the container.	All pipeline scripts were executed inside the Airflow container so that paths matched the container's filesystem, not the host.
Schema mismatch between the Gold layer files and the PostgreSQL tables.	The Gold layer schema was updated to match the target tables and PostgreSQL was reloaded to reflect the corrected structure.

17. Results and Outputs

The pipeline was executed end to end in Dockerised Airflow. All eleven tasks completed successfully. The implemented DAG follows the parallel SCD branch and final load_postgres → metadata → audit sequence documented in Figure 1.

Airflow execution confirmed that ingestion, all Medallion stages, both SCD tasks, transformation, Gold generation, PostgreSQL loading, Metadata and Audit completed without failure.

PostgreSQL verification confirmed seven tables: business_dataset (113,425 rows), customer_summary (96,219), product_summary (32,328), monthly_summary (25), seller_summary (3,095), customer_dimension (99,441) and product_dimension_history (32,951).

18. Advantages

- Fully automated pipeline eliminates manual, error-prone data processing.
- Medallion architecture provides clear separation between raw, cleaned and business-ready data.
- SCD Type 1 and Type 2 support both simple corrections and full historical dimension tracking.
- Incremental order processing is integrated into ingestion and avoids appending unchanged static source records.
- Docker containerisation makes the entire environment portable and reproducible on any machine.
- Airflow orchestration provides scheduling, retries, and clear visual monitoring of every task.
- One Metadata task and one final Audit task provide concise dataset and run-level traceability.
- Built entirely on open-source tools, keeping the solution cost-effective and vendor-independent.

19. Conclusion

This project demonstrates a complete Olist data engineering pipeline using Airflow, Docker, Pandas, Parquet and PostgreSQL. Raw CSV data moves through ingestion,

Bronze, Validation and Silver; SCD Type 1 and Type 2 then execute in parallel, business transformation waits for both, and Gold outputs are loaded into PostgreSQL. Incremental order logic is integrated into ingestion, and the workflow finishes with one Metadata task and one Audit task.

The resulting system is scalable, maintainable and fully traceable, and it illustrates how open-source technologies alone can be used to build a production-style data platform. The project provides a strong foundation that can be extended with additional data sources, more advanced transformations, real-time streaming ingestion, or deployment to a cloud-managed Airflow and PostgreSQL environment.

20. Project Repository

The complete source code for this project, including the Airflow DAG definitions, transformation scripts, Docker configuration, and audit/metadata modules, is available on GitHub:**https://github.com/Jayasrirajendiran/docker-data-pipeline**